

TECNICATURA UNIVERSITARIA EN PROGRAMACION

BASE DE DATOS II

Trabajo Practico - Unidad 3, Semana 5
Indices, vistas y vistas materializadas en Food Store
Flujo obligatorio con Kiro, OpenCode y Git

Food Store

Plan de indexado, vistas y vista materializada
Informe de mediciones + Anexos SQL (extractos)

Equipo / Estudiantes:	Matias Limina Nicolas Monjelardi Lautaro Aguero
Repositorio GitHub:	https://github.com/MatiasLimina/BaseDeDatos2.git
Materia / Unidad:	Base de Datos II - Unidad 3 Semana 1 (Semana 5 del TP)
Motor / Herramientas:	PostgreSQL 16+ Kiro (specs) OpenCode Git/GitHub
Protocolo seguridad:	Copia foodstore_copia, backup pg_dump y transacciones reversibles
Fecha / Estado:	05/09/2026 (Actualizado: 23/09/2026) Entregables completos con README y DUIA
Archivos incluidos:	README.md, indices.sql, views.sql, queries.sql, materializadas.sql, duia.md

Defensa oral: cada decision debe poder justificarse sin apoyo de IA en el momento (Regla de la catedra, pag. 6).

Indice

1. Presentacion y punto de partida	3
2. Parte A - Plan de indexado asistido por IA (consigna 4.1)	3
2.1 Consultas frecuentes (queries.sql)	3
2.2 Indices propuestos (indices.sql)	4
2.3 Mediciones EXPLAIN ANALYZE - Resumen ejecutivo	4
2.4 Consulta 1 - Historial por fecha (caso exitoso)	5
2.5 Consulta 2 - Ranking productos (planner ignora indice)	5
2.6 Consulta 3 - Detalle pedido (ya eficiente)	6
2.7 Costo en escrituras - INSERT 500 filas	6
2.8 Indice descartado por sobreindexacion	7
3. Parte B - Vistas para reportes (consigna 4.2)	7
3.1 Definicion de las 3 vistas (views.sql)	7
3.2 Criterio de seguridad (Requisito 2)	8
3.3 Verificacion de equivalencia EXCEPT 0 filas	8
4. Parte C - Vista materializada (consigna 4.3)	9
4.1 Definicion mv_facturacion_categoria_mes	9
4.2 Medicion comparativa (760 ms -> 0.020 ms)	9
4.3 Frecuencia de REFRESH y latencia	10
5. Flujo de trabajo Kiro -> OpenCode -> Git	10
6. Declaracion de Uso de IA (DUIA) - Bitacora	11
Anexos - Extractos SQL (literal, sin reescribir)	11

Nota: paginacion orientativa. El informe completo detallado (732 lineas) esta en TP3/informe_mediciones.md y se resume aqui con tablas y extractos literales para cumplir formato de entrega (consigna pag. 6) sin duplicar archivos completos.

1. Prese

El TP pone en juego los contenidos de Semana 5: tipos de indices, criterios de creacion, costo de mantenimiento, y vistas simples y materializadas como herramienta de simplificacion, seguridad y reutilizacion. Se trabaja sobre Food Store tomando como base el esquema, datos y consultas de semanas anteriores, sin modificar el modelo (consigna pag. 2, punto 3). La IA es motor primario: Kiro para especificar antes de generar, OpenCode como agente en terminal para producir SQL. El repo Git debe reflejar el flujo, no solo el resultado.

Esquema base (schema.sql - heredado)

5 tablas: categoria (id, nombre UNIQUE, activo), producto (id, nombre, precio NUMERIC(10,2), stock, activo, categoria_id FK RESTRICT), cliente (id, nombre, apellido, email UNIQUE, telefono, activo), pedido (id, fecha TIMESTAMPTZ, forma_pago forma_pago_enum, cliente_id FK), pedido_detalle (PK compuesta pedido_id+producto_id, cantidad, precio_unitario, subtotal, CHECKs, FKs RESTRICT). Tipos: BIGINT GENERATED ALWAYS AS IDENTITY, VARCHAR, NUMERIC(10,2), TIMESTAMPTZ, ENUM forma_pago_enum (EFECTIVO,TARJETA,TRANSFERENCIA,OTRO). Indices heredados Semana 3: idx_pedido_cliente_id ON pedido(cliente_id) e idx_producto_categoria_activo ON producto(categoria_id, activo). Volumen de prueba: ~621.199 filas pedido_detalle, 200.000 pedido, 50.005 producto, 3 categorias (mediciones reales pag. 6).

2. Parte A - Plan de indexado asistido por IA (Consigna 4.1)

Objetivo (pag. 3, punto 4.1): identificar ≥ 3 consultas con Seq Scan, especificar en Kiro (consulta, frecuencia, columnas filtro/JOIN/ORDER BY), proponer indice con OpenCode, medir EXPLAIN ANALYZE antes/después, medir costo en escrituras (INSERT detalle_pedido), y descartar al menos 1 por sobreindexacion. Entregable: indices.sql + informe_mediciones.md.

2.1 Consultas frecuentes - queries.sql (extractos literales)

Consulta 1 - Historial ventas por fecha (Alta, filtro fecha + forma_pago) - queries.sql:5

```
SELECT * FROM pedido
WHERE fecha BETWEEN '2023-01-01' AND '2023-12-31'
AND forma_pago = 'EFECTIVO';
```

Consulta 2 - Ranking Top 5 productos mas vendidos (Media, JOIN pd.producto_id, GROUP/ORDER) - queries.sql:13

```
SELECT p.nombre, SUM(pd.cantidad) AS total_vendido
FROM pedido_detalle pd JOIN producto p ON pd.producto_id = p.id
GROUP BY p.id ORDER BY total_vendido DESC LIMIT 5;
```

Consulta 3 - Detalle pedido ordenado por subtotal (Media, filtro pedido_id + ORDER BY subtotal) - queries.sql:24

```
SELECT * FROM pedido_detalle WHERE pedido_id = 123 ORDER BY subtotal DESC;
```

Bloque escritura - INSERT 500 filas pedido_detalle - queries.sql:28

```
DO $$ BEGIN FOR i IN 1..500 LOOP
  INSERT INTO pedido_detalle (pedido_id, producto_id, cantidad, precio_unitario, subtotal)
  VALUES (i, (i%100)+1, FLOOR(RANDOM()*10)+1, ROUND((RANDOM()*100)::numeric,2), 0);
END LOOP; END $$;
```

2.2 Indices propuestos - indices.sql (extractos literales)

Indice 1 - indices.sql:5

```
CREATE INDEX idx_pedido_fecha ON pedido(fecha); -- B-tree rango fecha
```

Indice 2 - indices.sql:11

```
CREATE INDEX idx_detalle_producto_id ON pedido_detalle(producto_id); -- B-tree 2do campo PK no indexado solo
```

Indice 3 - indices.sql:17

```
CREATE INDEX idx_detalle_subtotal ON pedido_detalle(pedido_id, subtotal DESC); -- compuesto evita sort
```

Descartado comentado - indices.sql:19

```
-- Indice descartado ON pedido(forma_pago) -- baja cardinalidad 4 valores ENUM
-- Justif: Seq Scan preferido, redundante, solo util parcial WHERE forma_pago='TARJETA' si <5%
```

2.3 Resumen ejecutivo de mediciones (informe_mediciones.md:45)

#	Consulta	Plan ANTES	Plan DESPUES	Uso?	Conclusion
1	Hist.por fecha	Parallel SeqScan 289.652 ms	IndexScan idx_pedido_fecha 0.018 ms	Si	Aceptado (~16.000x)
2	Ranking Top5	SeqScan+HashJoin 423.892 ms	SeqScan+HashJoin 218.262 ms	No	Descartado (Sobreindexacion)
3	Detalle pedido	IdxScan PK 0.055 ms (no SeqScan)	IdxScan idx_detalle_subtotal 0.102 ms	Cambio	Descartado (+85% overhead)
4	Forma de pago	SeqScan	-- (Baja cardinalidad 4 ENUM)	-	Descartado por baja select.
-	INSERT 500 filas	0.313 s	0.058 s	-	Paradoja warm cache

2.4 Consulta 1 - Caso exitoso (Alta selectividad rango fecha)

Frecuencia: Alta (reporte diario/semanal). Columnas: pedido.fecha BETWEEN + forma_pago ENUM. Sin indice en fecha, unico idx era idx_pedido_cliente_id, inutilizable -> Parallel Seq Scan. Indice idx_pedido_fecha B-tree ASC cubre rangos; forma_pago se filtra post-Index Scan (baja cardinalidad no justifica compuesto).

EXPLAIN ANALYZE - Anotacion mediciones.txt:2 / informe_mediciones.md:2.4

```
ANTES: Parallel Seq Scan on pedido (cost=0.00..3725.83 rows=1) actual 164.437ms loops=2 | Planning 2.810 ms | Execution 289.652 ms
DESPUES: Index Scan using idx_pedido_fecha (cost=0.29..8.32 rows=1) actual 0.004ms | Planning 1.669 ms | Execution 0.018 ms
```

Metrica	Antes	Despues	Delta
Access	Parallel Seq Scan	Index Scan idx_pedido_fecha	Cambio estrategia
Cost	0.00..3725.83	0.29..8.32	-99.8%
Planning	2.810 ms	1.669 ms	-1.141 ms
Execution	289.652 ms	0.018 ms	-289.634 ms (~16000x)

Analisis: unica hipotesis verificada. Caída de cost y tiempo no explicable por cache; cambio de plan confirma adopcion. Decision: ACEPTADO.

2.5 Consulta 2 - Planner ignora indice (Sobreindexacion)

Frecuencia: Media (ranking semanal). PK pedido_detalle es (pedido_id, producto_id); 2do campo no usable solo para JOIN/GROUP BY producto_id. Se propuso idx_detalle_producto_id para acelerar el Hash Join.

EXPLAIN ANALYZE - Anotacion mediciones.txt:6 / informe_mediciones.md:3.4

```
ANTES: Sort top-N heapsort -> HashAggregate -> Hash Join (cost 1641..14701 rows 623k) -> SeqScan pd 0.028..27.892 (621199) -> SeqScan producto 0.011..199.901 | Planning 11.944 ms | Exec 423.892 ms
DESPUES: Mismo Sort/HashAggregate/Hash Join -> SeqScan pd 0.014..25.382 (621199) -> SeqScan producto 0.006..4.030 | Planning 0.179 ms | Exec 218.262 ms
```

Metrica	Antes	Despues	Delta
Access pd	Seq Scan	Seq Scan (idx no usado)	Sin cambio
Cost Sort	19147..19272	19113..19238	-0.2% marginal

Planning	11.944 ms	0.179 ms	-11.765 ms cache catalogo
Execution	423.892 ms	218.262 ms	-205 ms (-48.5%)
Rows pd	621199	621199	0

Analisis honesto: la mejora NO se debe al indice (el plan mantiene Seq Scan). Con 621k filas que deben agregarse en su totalidad, el costo de random I/O mediante Index Scan es mayor que un Seq Scan secuencial. La caida en tiempo se debio a warm cache. Decision: DESCARTADO POR SOBREINDEXACION. Mantener un indice que el optimizador descarta sistematicamente solo anadiria costo de mantenimiento en escrituras sin beneficio de lectura.

2.6 Consulta 3 - Diagnostico corregido: ya eficiente via PK (Sobreindexacion)

Frecuencia: Media (detalle pedido en UI/comprobante). Filtro pedido_id + ORDER BY subtotal DESC. Correccion de diagnostico: la consulta NO resolvia con Seq Scan; ya utilizaba Index Scan gracias a la PK compuesta (pedido_id, producto_id) donde pedido_id es columna lider.

EXPLAIN ANALYZE - Anotacion mediciones.txt:15 / informe mediciones.md:4.4

ANTES: Index Scan using pk_pedido_detalle (cost 0.42..11.98 rows3) actual 0.008..0.022 rows1 | Planning 0.156 ms | Execution 0.055 ms
 DESPUES: Index Scan using idx_detalle_subtotal (cost 0.42..11.98 rows3) actual 0.087..0.088 rows1 | Planning 0.097 ms | Execution 0.102 ms

Metrica	Antes	Despues	Delta
Access	IdxScan pk_pedido_detalle	IdxScan idx_detalle_subtotal	Cambio indice
Cost	0.42..11.98	0.42..11.98	Identico
Planning	0.156 ms	0.097 ms	-0.059 ms
Execution	0.055 ms	0.102 ms	+0.047 ms (+85% overhead)

Analisis honesto: la consulta ya era sub-milisegundo (0.055 ms) por clave primaria. El nuevo indice no reduce costo e incrementa el tiempo (+85% overhead) sin beneficio real para particiones pequenas. Decision: DESCARTADO POR SOBREINDEXACION. Se evita crear un objeto secundario redundante.

2.7 Costo en escrituras - INSERT 500 filas en pedido_detalle

Script medido - Anotacion mediciones.txt:19

```
DO $$ BEGIN FOR i IN 1..500 LOOP INSERT INTO pedido_detalle (...) VALUES (i, (i%100)+1, ...); END LOOP; END $$; --
queries.sql:28
```

Fase	Tiempo (Execute)	Indices en pedido_detalle
ANTES (sin idx propuestos)	0.313 s	PK pk_pedido_detalle + FKs
DESPUES (+2 idx nuevos)	0.058 s	PK + idx_detalle_producto_id + idx_detalle_subtotal

Delta -0.255s (-81%) paradójico: anadir indices deberia encarecer (actualizar B-tree + WAL). Inversion por warm cache: primera ejecucion con shared_buffers frio (lectura disco FKs e indices), segunda reuso paginas en memoria + metadatos (Planning 11.9ms->0.179ms evidencia cache). Sin DISCARD/CHECKPOINT/reinicio. En produccion frio se espera +5-15% por indice, no -81%. Medicion honesta no concluyente; documenta comportamiento cache. Para rigor: CHECKPOINT; DISCARD PLANS; repetir Nx y promediar con EXPLAIN BUFFERS.

2.8 Indice descartado por sobreindexacion (consigna 4.1.p6)

Propuesta descartada - indices.sql:19 + Anotacion mediciones.txt:44

```
-- CREATE INDEX idx_pedido_forma_pago ON pedido(forma_pago); -- DESCARTADO
```

Columna forma_pago_enum con 4 valores (EFECTIVO,TARJETA,TRANSFERENCIA,OTRO) - schema.sql:20. Baja cardinalidad => selectividad ~25% por valor; B-tree no reduce costo vs SeqScan+Filter (acceso aleatorio mas caro). PostgreSQL prefiere SeqScan (verificado en C1). Mantenimiento (WAL, page split) sin beneficio. Solo tendria sentido como indice parcial WHERE forma_pago='TARJETA' si <5% y consultas exclusivas por ese valor. Decision: DESCARTADO, bloque comentado, justificado por escrito en informe mediciones.md:6 y DUIA.

3. Parte B - Vistas para reportes (Consigna 4.2)

Objetivo: especificar en Kiro >=3 vistas (productos vigentes con categoria, pedidos con datos usuario, detalle pedido con nombre producto) con columnas exactas, filtro vigencia y columna oculta por seguridad. Generar con OpenCode y verificar equivalencia EXCEPT. Al menos 1 vista debe ocultar contrasena (patron seguridad). Entregable: views.sql + nota equivalencia en informe. Spec: Proyecto_Integrador/specs/spec_punto_4.2/requirements.md (EARS).

3.1 Definicion de las 3 vistas - views.sql (extractos literales)

Vista 1 - Requisito 1 - views.sql:20 - Columnas: id, nombre, precio, stock, nombre_categoria, created_at

```
CREATE OR REPLACE VIEW vw_productos_vigentes AS
SELECT p.id, p.nombre, p.precio, p.stock, c.nombre AS nombre_categoria, p.created_at
FROM producto p JOIN categoria c ON p.categoria_id=c.id
WHERE p.activo=TRUE AND c.activo=TRUE;
```

Vista 2 - Requisito 2 - views.sql:43 - Columnas: pedido_id, fecha, forma_pago, cliente_id, nombre, apellido, activo

```
CREATE OR REPLACE VIEW vw_pedidos_cliente AS
SELECT p.id AS pedido_id, p.fecha, p.forma_pago, c.id AS cliente_id, c.nombre, c.apellido, c.activo
-- Excluidos: c.email, c.telefono, c.created_at (+futura contrasena)
FROM pedido p JOIN cliente c ON p.cliente_id=c.id;
```

Vista 3 - Requisito 3 - views.sql:77 - Columnas: pedido_id, nombre_producto, cantidad, precio_unitario, subtotal

```
CREATE OR REPLACE VIEW vw_detalle_pedido AS
SELECT pd.pedido_id, pr.nombre AS nombre_producto, pd.cantidad, pd.precio_unitario, pd.subtotal
FROM pedido_detalle pd JOIN producto pr ON pd.producto_id=pr.id;
```

3.2 Criterio de seguridad y prueba de roles DCL (Consigna 4.2.4)

Patron: exponer datos de cliente sin columnas de contacto sensibles (email, telefono, created_at). Se implemento y probo la asignacion de privilegios mediante un rol restringido en views.sql:

Prueba DCL con rol restringido - views.sql:190

```
CREATE ROLE rol_reportes_foodstore WITH LOGIN PASSWORD 'AuditorPassword2026!';
GRANT SELECT ON vw_pedidos_cliente TO rol_reportes_foodstore;
REVOKE ALL ON cliente, pedido FROM rol_reportes_foodstore;
-- Prueba en sesion:
SET ROLE rol_reportes_foodstore;
SELECT * FROM vw_pedidos_cliente LIMIT 1; -- EXITOSO (solo columnas seguras)
SELECT * FROM cliente LIMIT 1;           -- ERROR: permission denied for table cliente
RESET ROLE;
```

3.3 Verificacion de equivalencia - EXCEPT complementado con COUNT(*)

Metodologia rigurosa: EXCEPT aplica DISTINCT implicito (semantica de conjuntos). Para evitar falsos positivos ante duplicados, se complementa con la comparacion exacta de COUNT(*):

Vista	Diferencia EXCEPT	COUNT(*) Vista	COUNT(*) Manual	Equivalencia
vw_productos_vigentes	0 filas	49.850 filas	49.850 filas	Multiconjunto OK
vw_pedidos_cliente	0 filas	200.000 filas	200.000 filas	Multiconjunto OK
vw_detalle_pedido	0 filas	621.199 filas	621.199 filas	Multiconjunto OK

Ambas direcciones de EXCEPT retornaron 0 filas y la cardinalidad de tuplas coincide al 100%, descartando cualquier discrepancia de duplicidad.

4. Parte C - Vista materializada (Consigna 4.3)

Elegir reporte agregado costoso (ej. facturacion por categoria y mes) y crear vista materializada WITH DATA + indice unico para REFRESH CONCURRENTLY. Medir tiempo vista vs consulta original. Documentar frecuencia REFRESH y latencia. Entregable: materializadas.sql + medicion en informe. Spec: Proyecto_Integrador/specs/spec_punto_4.3/requirements.md (5 requisitos).

4.1 Definicion - materializadas.sql (extractos literales)

Vista materializada - materializadas.sql:50 - 4 JOINS + SUM + COUNT DISTINCT sobre ~621k filas

```
CREATE MATERIALIZED VIEW mv_facturacion_categoria_mes AS
SELECT c.nombre AS categoria, DATE_TRUNC('month',p.fecha) AS mes,
      COUNT(DISTINCT p.id) AS total_pedidos, SUM(pd.subtotal) AS facturacion_total
FROM pedido_detalle pd JOIN pedido p ON pd.pedido_id=p.id
JOIN producto pr ON pd.producto_id=pr.id JOIN categoria c ON pr.categoria_id=c.id
GROUP BY c.nombre, DATE_TRUNC('month',p.fecha)
ORDER BY mes DESC, facturacion_total DESC
WITH DATA;
```

Indice UNIQUE - materializadas.sql:79 - Requisito 2

```
CREATE UNIQUE INDEX idx_mv_facturacion_categoria_mes ON mv_facturacion_categoria_mes(categoria, mes);
-- Prerrequisito REFRESH CONCURRENTLY: sin UNIQUE falla "cannot refresh concurrently without a unique index"; no es indice de
busqueda primario
```

Refresco concurrente - materializadas.sql:112

```
REFRESH MATERIALIZED VIEW CONCURRENTLY mv_facturacion_categoria_mes;
-- pg_cron diario 04:00: SELECT cron.schedule('refresh-mv','0 4 * * *', $$REFRESH ...$$);
```

4.2 Medicion comparativa - EXPLAIN (ANALYZE, BUFFERS) reales

Fuente: anotaciones_vistas_materializadas.txt - base 621.199 pd + 200.000 pedido + 50.005 producto + 3 categorias. Dos

bloques materializadas.sql:12 (ANTES) y :94 (DESPUES).

Plan Consulta Original (4 JOINS + agregacion)

ANTES: Incremental Sort (cost66612..154189) SortKey date_trunc month DESC, sum DESC | GroupAggregate | Gather Merge 2 workers
Sort external merge Disk 8880kB (temp read3170 written3179) -> HashJoin pr.categoria_id=c.id -> HashJoin pd.producto_id=pr.id
-> Parallel HashJoin pd.pedido_id=p.id -> Parallel SeqScan pd 3625 hit+1568 read / Parallel SeqScan pedido 1667 | Buffers
shared hit6894 read1568 | Planning 67.035 ms | Execution 760.827 ms

Plan SELECT * FROM mv_facturacion_categoria_mes

DESPUES: Seq Scan on mv_facturacion_categoria_mes (cost0.00..1.24 rows24 width226) actual 0.009..0.010 rows24 | Buffers
shared hit1 | Planning Buffers hit21 read1 dirtied3 | Planning 1.567 ms | Execution 0.020 ms

Metrica	Consulta Original	SELECT * FROM mv	Delta
Metodo	Incremental Sort->GroupAgg->GatherMerge->Sort ext.->HashJoin x3	SeqScan mv (24 filas material.)	Elimina JOINS/sorts
Cost	66612..154189	0.00..1.24	-99.99%
Rows	621199 leidas /24 agrup.	24 materializadas	1 por (cat,mes)
Planning	67.035 ms	1.567 ms	-65.468 ms (-97.7%)
Execution	760.827 ms	0.020 ms	-760.807 ms (~38041x)
Buffers shared	hit6894 read1568	hit1 (+plan hit21)	-99.98%
Buffers temp	read3170 written3179	0	Elimina I/O temp

Analisis: Execution 760ms->0.020ms (~38k x) confirma hipotesis Requisito 3.3. Consulta paga 4 JOINS hasheados, 2 sorts (8880kB disco) y Gather Merge paralelo; vista reemplaza por SeqScan 24 filas (1 por categoria-mes) sin temp y Planning -97.7%. A mayor pd, mayor brecha (costo original lineal, vista constante). Caso exitoso complementario a Parte A donde 2/3 indices fueron ignorados/marginales.

4.3 Frecuencia de REFRESH y latencia (Requisito 4)

Aspecto	Definicion
Frecuencia propuesta	>=1 vez/dia, madrugada o cierre jornada (Requisito 4.1)
Justificacion	Reporte de Gestion historico (cierre mes, tendencias) no requiere tiempo real
Latencia maxima	~24h con refresco diario (transaccion post-REFRESH invisible hasta siguiente ciclo) 4.2
Ejemplo	Pedido 05/09 10:00 con refresh 04:00 visible recién 06/09 04:00
Adecuada para	Reporte_Gestion (Si) Dashboard_Tiempo_Real (No) - requiere seg/min 4.3
Alta carga	Opcion 4-6h (tradeoff: latencia 4-6h vs costo CPU/I/O re-materializar 4 JOINS) 4.4
Comando exacto	REFRESH MATERIALIZED VIEW CONCURRENTLY mv_facturacion_categoria_mes; 4.5
Programacion	pg_cron 0 4 * * * o script nocturno (sin bloqueo lecturas gracias a UNIQUE)
Limite	<1h evaluar reemplazar por vista ordinaria con indices o tabla agregada por triggers/pg_ivm 4.6

Ejemplo pg_cron - informe Mediciones.md:9.4.5

```
SELECT cron.schedule('refresh-mv-facturacion','0 4 * * *', $$REFRESH MATERIALIZED VIEW CONCURRENTLY mv_facturacion_categoria_mes$$);
```

5. Flujo de trabajo obligatorio: Kiro, OpenCode y Git (Consigna pag. 5)

Secuencia exigida por catedra para cada pieza (indice, vista, mat. view): 1) Especificar en Kiro (que/criterio) -> archivo en specs/ 2) Generar con OpenCode a partir del spec (leer linea por linea, no delegar decision) 3) Probar en copia/transaccion reversible con respaldo 4) Versionar en Git commits separados descriptivos (ej: "Indice pedido(fecha) - reduce SeqScan en reporte mensual") con diff mostrable en defensa. El historial documenta proceso, no solo entrega final.

Paso	Herramienta	Artefacto en repo
1 Especificar	Kiro	specs/spec_punto_4_1.md, specs/spec_punto_4.2/requirements.md, specs/spec_punto_4.3/requirements.md
2 Generar	OpenCode	TP3/queries.sql, indices.sql, views.sql, materializadas.sql
3 Probar	psql EXPLAIN	Anotacion_mediciones.txt, anotaciones_vistas_materializadas.txt
4 Versionar	Git	Commits: idx_pedido_fecha, idx_detalle_*, vistas, mv + informe_mediciones.md

Evidencia en entrega: specs conservados en Proyecto_Integrador/specs/, SQL generado leído linea por linea y probado antes de crear indices/vistas, mediciones registradas con EXPLAIN (ANALYZE, BUFFERS) y verificaciones EXCEPT 0 filas.

6. Declaracion de Uso de IA (DUIA) - Bitacora (Consigna pag. 5)

Archivo duia.md = bitacora integrada en TP3/informe_mediciones.md (Notas DUIA por parte, pag. 5 requisito minimo: caso sobreindexacion Parte A y equivalencia Parte B). Cada interaccion registra: herramienta, prompt/spec, propuesta IA, aceptado/modificado/descartado con justificacion tecnica.

Parte	Herramienta	Que genero	Que se acepto	Que se descarto/modifico y por que
A	OpenCode muse-spark-1	Plan indexado 3 idx + descartado Estructura de mediciones (5KB Alta)	Desempeño de 15KB Alta	baja card. 4 ENUM, planner SeqScan, costo mant. no compensa (informe:6)
B	Kiro (specs) + OpenCode	requirements.md + 3 vistas + 3 DCLs	EXPLAIN ANALYZE, Justificaciones SQL y planes de ejecución	cliente: sin contrasena en esquema, documentado con email/telefono +
C	OpenCode muse-spark	materializadas.sql + shared_buffers exactos, WITH DATA, REFRESH_CONCURRENTLY, Latencia de Dato	SQL de Escalado	orders reemplazados por valores reales 67->1.5ms, 760->0.02ms (informe:6)

Verificaciones: contraste manual EXPLAIN ANTES/DESPUES vs Anotacion_mediciones.txt; contrato columnas/JOINS/GROUP BY/ORDER BY/WITH DATA/nombre indice vs requirements.md; EXCEPT cubren mismas columnas que vistas; shared_buffers/cache documentado honestamente (casos donde planner ignoro indice). Registro sobreindexacion: IA sugirio indice forma_pago, descartado por criterio humano (Anotacion:44-52, informe:6, DUIA A).

Anexos - Extractos SQL literales (sin reescribir, solo extractos)

Para cumplir formato entrega (consigna pag. 6) se incluyen extractos literales con referencia ruta:linea. Archivos completos en repo: TP3/queries.sql, indices.sql, views.sql, materializadas.sql y Proyecto_Integrador/database/ espejos. No se modifico schema.sql (5 tablas, R1-R7).

A. queries.sql - 3 consultas + bloque escritura (TP3/queries.sql:1)

Extracto resumido - ver archivo completo TP3/queries.sql:1-35

```
-- Cons.1: SELECT * FROM pedido WHERE fecha BETWEEN '2023-01-01' AND '2023-12-31' AND forma_pago='EFECTIVO'; -- Alta
-- Cons.2: SELECT p.nombre, SUM(pd.cantidad) FROM pedido_detalle pd JOIN producto p GROUP BY p.id ORDER BY SUM DESC LIMIT 5;
-- Media
-- Cons.3: SELECT * FROM pedido_detalle WHERE pedido_id=123 ORDER BY subtotal DESC; -- Media
-- INSERT: DO $$ FOR i IN 1..500 LOOP INSERT INTO pedido_detalle ... END LOOP; END $$;
```

B. indices.sql - 1 ACEPTADO + 3 DESCARTADOS por sobreindexacion

Extracto - TP3/indices.sql:1-37 + mediciones EXPLAIN ANALYZE

```
CREATE INDEX idx_pedido_fecha ON pedido(fecha); -- ACEPTADO: B-tree rango (~16.000x)
-- DESCARTADO 1: idx_detalle_producto_id -- Planner ignoro (SeqScan); sobreindexacion
-- DESCARTADO 2: idx_detalle_subtotal -- Ya eficiente por PK; +85% overhead; sobreindexacion
-- DESCARTADO 3: idx_pedido_forma_pago -- Baja cardinalidad (4 valores ENUM)
```

C. views.sql - 3 vistas + verificacion + rol DCL (TP3/views.sql:1)

Extracto definiciones - TP3/views.sql:12-82

```
vw_productos_vigentes: SELECT p.id,p.nombre,p.precio,p.stock,c.nombre AS nombre_categoria FROM producto p JOIN categoria c
WHERE p.activo AND c.activo
vw_pedidos_cliente: SELECT p.id AS pedido_id,p.fecha,p.forma_pago,c.id AS cliente_id,c.nombre,c.apellido,c.activo FROM pedido
p JOIN cliente c -- omite email/telefono/created_at
vw_detalle_pedido: SELECT pd.pedido_id,pr.nombre AS nombre_producto,pd.cantidad,pd.precio_unitario,pd.subtotal FROM
pedido_detalle pd JOIN producto pr
```

Verificacion Requisito 4 y 4.2.4 - views.sql:85-220

```
(SELECT ... FROM vw_productos_vigentes) EXCEPT (SELECT ... FROM producto JOIN categoria WHERE ...) -- 0 filas
Validacion COUNT(*): 49.850 = 49.850 | Prueba DCL: GRANT SELECT a rol_reportes_foodstore y REVOKE en cliente/pedido (bloqueo
verificado).
```

D. materializadas.sql - vista + indice + EXPLAIN (TP3/materializadas.sql:1)

Extracto completo - TP3/materializadas.sql:1-115

```
EXPLAIN (ANALYZE,BUFFERS) SELECT c.nombre, DATE_TRUNC('month',p.fecha), COUNT(DISTINCT p.id), SUM(pd.subtotal) FROM ... --
ANTES 67ms/760ms
CREATE MATERIALIZED VIEW mv_facturacion_categoria_mes AS SELECT ... GROUP BY c.nombre, DATE_TRUNC ... ORDER BY mes DESC WITH
DATA; -- :50
CREATE UNIQUE INDEX idx_mv_facturacion_categoria_mes ON mv_facturacion_categoria_mes(categoria,mes); -- :79
EXPLAIN (ANALYZE,BUFFERS) SELECT * FROM mv_facturacion_categoria_mes; -- DESPUES 1.5ms/0.020ms
-- REFRESH MATERIALIZED VIEW CONCURRENTLY mv_facturacion_categoria_mes; -- :112
```

E. Specs - Referencias

Kiro specs conservados en repo (carpeta specs/ como exige pag. 5): Proyecto_Integrador/specs/spec_punto_4_1.md (5.3kB, Plan de Indexado), Proyecto_Integrador/specs/spec_punto_4.2/requirements.md (Parte B, 3 requisitos EARS), Proyecto_Integrador/specs/spec_punto_4.3/requirements.md (Parte C, 5 requisitos EARS, Glosario Vista_Materializada, WITH_DATA, REFRESH_CONCURRENTLY, Latencia_de_Dato). Cada spec incluye Introduction, Glossary, Requirements y criterios de aceptacion que OpenCode uso como prompt.

F. Guia de reproduccion y protocolo de seguridad (Ver README.md)

Flujo reproducible - protocolo de seguridad: backup previo y transacciones reversibles

```
# 1. Respaldo preventivo y copia de trabajo (protocolo_seguridad.md)
pg_dump -U postgres -d foodstore -F c -f backup_foodstore_pre_tp3.backup
createdb -U postgres -T foodstore foodstore_copia
# 2. Restaurar esquema y volumen de datos
psql -d foodstore_copia -f Proyecto_Integrador/database/schema.sql
psql -d foodstore_copia -f Proyecto_Integrador/database/seed.sql
# 3. Pruebas de queries e indices
psql -d foodstore_copia -f TP3/queries.sql
psql -d foodstore_copia -f TP3/indices.sql
# 4. Vistas y prueba DCL de rol
psql -d foodstore_copia -f TP3/views.sql
# 5. Vista materializada con medicion y refresco concurrente
psql -d foodstore_copia -f TP3/materializadas.sql
```

7. Cierre y criterios de evaluacion

Al finalizar, Food Store cuenta con plan de indexado justificado con datos (no intuicion) y conjunto de vistas base para objetos programables de Semana 6 (procedimientos, funciones, triggers) - pag. 6 punto 10. No eliminar archivos: se reutilizan. Criterios orientativos (pag. 6 punto 9): diseno de plan con mediciones honestas (casos donde indice no ayuda), vistas que simplifican/protegen/estandarizan, vista materializada con WITH DATA + UNIQUE + medicion, flujo Kiro/OpenCode/Git verificable, y defensa oral sin apoyo IA (explicar por que se creo/descarto cada indice y que garantiza cada vista). Entregar script que no se puede explicar equivale a no haber realizado el trabajo.

Repositorio GitHub: <https://github.com/MatiasLimina/BaseDeDatos2.git> con historial de commits y diff por commit. Autores: Matias Limina, Nicolas Monjelardi, Lautaro Agüero. Entregables completos con README.md, duia.md, informe_mediciones.md y PDF.